Trabalho Colaborativo com Github

Roteiro de Desenvolvimento: Totem Hospitalar "Viver Bem"

Este projeto simula um fluxo real de engenharia de software. O sucesso depende da comunicação e do respeito às ramificações (branches).

📂 Estrutura de Pastas Esperada

Para que não haja conflitos de arquivos, cada aluno criará arquivos específicos no visual code. O Líder deve garantir que ninguém saia dessa estrutura:

```
/totem-hospitalar
│
├── main.py                # Arquivo principal que une tudo (Editado pelo Líder)
├── /models                # Pasta para as classes do sistema
│   ├── __init__.py        # Arquivo vazio (marcador de pacote)
│   ├── paciente.py        # Criado pelo Dev Um
│   ├── classificador.py    # Criado pelo Dev Um
│   ├── triagem.py         # Criado pelo Dev Um
│   ├── menu.py            # Criado pelo Dev Dois
│   ├── gerador_senha.py   # Criado pelo Dev Dois
│   └── impressora.py      # Criado pelo Dev Dois
└── README.md              # Documentação do projeto
```

🛠️ Cronograma de Ações Reais

Fase 1: Setup e Infraestrutura (Início de tudo)

1. **Líder (Configuração):** Cria o repositório no GitHub, adiciona os colegas como colaboradores e faz o **Clone** no GitHub Desktop. Sobe a pasta `/models` com o `__init__.py` e o `main.py` vazio.
2. **Dev Um e Dois(Sincronia):** Abrem o GitHub Desktop e fazem o **Clone** do repositório para suas máquinas.

Fase 2: Sprint 1 - O Alicerce dos Dados

- **Dev Um:** Cria a Branch `feat-paciente`.
  - **Ação:** Desenvolver a classe `Paciente` (nome, idade, pcd) em `models/paciente.py`.
  - Dá o **Commit**, **Push** e abre o **Pull Request (PR)**.
- **Dev Dois:** Cria a Branch `feat-menu`.
  - **Ação:** Desenvolver a classe `Menu` em `models/menu.py` que lê as entradas do teclado para o serviço desejado.
  - Dá o **Commit**, **Push** e abre o **PR**.
- **Líder (Review 1):** Entra no GitHub, revisa o código dos dois.
  - **Teste:** O código roda? A classe `Paciente` armazena os dados? O `Menu` exibe as opções corretamente?
  - **Ação:** Se estiver ok, faz o **Merge** de ambos.
- **TODOS:** No GitHub Desktop, cliquem em **Fetch Origin** e depois em **Pull** para atualizar a sua `main` local com o trabalho dos colegas.

Fase 3: Sprint 2 - A Inteligência do Negócio

- **Dev Um:** Cria a Branch `feat-classificador`.
  - **Ação:** Desenvolver `models/classificador.py`. Esta classe recebe um objeto `Paciente` e decide se ele é "Prioritário" ou "Normal" baseado na idade e deficiência.
  - Dá o **Commit**, **Push** e abre o **PR**.

- **Dev Dois:** Cria a Branch `feat-senha`.
  - **Ação:** Desenvolver `models/gerador_senha.py`. Classe que cria a numeração (ex: G-001 para geral e P-001 para prioridade).
  - Dá o **Commit**, **Push** e abre o **PR**.
- **Líder (Review 2): Ação:** Baixa as branches, testa se o `Classificador` está conversando bem com a classe `Paciente`. Se estiver tudo certo, faz o **Merge**.
- **TODOS:** Repetem o processo de **Fetch e Pull** para terem o código novo.

Fase 4: Sprint 3 - Fluxo de Emergência e Saída

- **Dev Um:** Cria a Branch `feat-triagem`.
  - **Ação:** Desenvolver `models/triagem.py`. Se o paciente escolher "Emergência", o sistema deve ignorar senhas e exibir um alerta: "ENCAMINHAR IMEDIATAMENTE AO BOX MÉDICO".
  - Dá o **Commit**, **Push** e abre o **PR**.
- **Dev Dois:** Cria a Branch `feat-impressora`.
  - **Ação:** Desenvolver `models/impressora.py`. Um módulo que recebe os dados finais e "imprime" um ticket bonito no console com a data e o serviço.
  - Dá o **Commit**, **Push** e abre o **PR**.
- **Líder (Integração Final):** Faz o **Merge** final das últimas branches.
  - **Ação Final:** Abre o arquivo `main.py`. Importa todas as classes desenvolvidas pelos colegas. Cria o loop principal (`while True`) que faz o totem funcionar: recebe o paciente → passa pelo menu → gera senha/triagem → imprime ticket.

🚨 Alertas de Dinâmica

> ⚠️ **NÃO ATRAVESSE O SAMBA:** Nunca abra o arquivo que o colega está editando. Se você precisa de uma informação do arquivo dele, espere o Líder aprovar o Merge e dê um **Pull** no seu computador.
>
> ⚠️ **MENSAGENS REAIS:** Em vez de "terminei", use "Adicionada lógica de verificação de idade para prioridade".
>
> ⚠️ **LÍDER NÃO É BABÁ:** O Líder não deve escrever o código para o colega. Se estiver errado, ele comenta no PR: "O método x está com erro de sintaxe, corrija para eu aprovar".
>
> ⚠️ **BRANCH É VIDA:** Nunca faça commits na `main`. Se você fizer isso, o Git vai bloquear seu colega de subir o código dele.

Saídas esperadas:

O código deve imprimir os seguintes resultados:

🏥 Exemplo 1: Paciente Padrão (Consulta)

Cenário: Usuário jovem, sem deficiência, buscando uma consulta de rotina.

```
==========================================
      HOSPITAL VIVER BEM - AUTOATENDIMENTO
==========================================

[CADASTRO DE PACIENTE]
Nome: Lorrany Marim
Idade: 31
Possui deficiência? (S/N): N

[MENU DE SERVIÇOS]
1. Consulta
2. Exame
```

```
3. Emergência
Escolha a opção: 1

-----------------------------------------
            TICKET DE ATENDIMENTO
-----------------------------------------
PACIENTE: Lorrany Marim
TIPO: Geral
SERVIÇO: Consulta
SENHA: G-001
-----------------------------------------
Aguarde ser chamado no painel central.
```

👨 Exemplo 2: Paciente Prioritário (Exame)

Cenário: Usuário acima de 60 anos ou com deficiência. O sistema deve reconhecer a prioridade automaticamente.

```
=========================================
      HOSPITAL VIVER BEM - AUTOATENDIMENTO
=========================================

[CADASTRO DE PACIENTE]
Nome: João Guimarães
Idade: 72
Possui deficiência? (S/N): N

[MENU DE SERVIÇOS]
1. Consulta
2. Exame
3. Emergência
Escolha a opção: 2

-----------------------------------------
            TICKET DE ATENDIMENTO
-----------------------------------------
PACIENTE: João Guimarães
TIPO: PRIORITÁRIO (Lei 10.048)
SERVIÇO: Exame
SENHA: P-001
-----------------------------------------

Aguarde ser chamado no painel central.
```

🚨 Exemplo 3: Caso de Emergência

Cenário: Independente da idade, o sistema identifica que o risco é imediato e não gera senha de fila comum.

```
=========================================
      HOSPITAL VIVER BEM - AUTOATENDIMENTO
=========================================

[CADASTRO DE PACIENTE]
Nome: Woody Batista
Idade: 25
Possui deficiência? (S/N): S

[MENU DE SERVIÇOS]
1. Consulta
```

```
2. Exame
3. Emergência
Escolha a opção: 3

-----------------------------------------
          ALERTA DE EMERGÊNCIA
-----------------------------------------
PACIENTE: Woody Batista
STATUS: RISCO IMEDIATO

>>> ATENÇÃO: Encaminhe-se IMEDIATAMENTE
    ao balcão de triagem médica.
    Não é necessário aguardar senha.
-----------------------------------------
```

Ação Git	O que faz na prática?	Quem faz?
Clone	"Copia" o projeto da nuvem para o PC.	Todos (uma vez)
Fetch / Pull	"Puxa" as novidades que os colegas fizeram.	Todos (sempre)
New Branch	Cria uma área isolada para não quebrar o código principal.	Devs
Commit	Cria um "ponto de salvamento" do que você escreveu.	Devs
Push	"Sobe" os seus salvamentos para a internet.	Devs
Pull Request	Avisa: "Terminei minha parte, pode revisar?"	Devs
Merge	"Mistura" o código aprovado na versão oficial.	Líder

Se não souber realizar a ação, pesquise como fazer cada utilizando o Git Desktop, não é necessário realizar via terminal CMD.